



Programmation fonctionnelle (OCAML)

Mouhamadou GAYE

Enseignant-Chercheur
Département d'Informatique
Master en Informatique
Option Génie Logiciel

16 juin 2025



Chapitre 3 : Modules et Classes



- OCaml ajoute une couche orientée objet et un système de modules au langage de base CAML.
- Le système comprend un compilateur produisant du code natif de haute performance (ocamlopt) pour 9 architectures de microprocesseurs.
- Il offre également des générateurs d'analyseurs lexicaux (ocamllex) et syntaxiques (ocamlyacc).



- Un module regroupe un ensemble de définitions apparentées



- Un module regroupe un ensemble de définitions apparentées
 - de valeurs (let)



- Un module regroupe un ensemble de définitions apparentées
 - de valeurs (let)
 - de types (type)



- Un module regroupe un ensemble de définitions apparentées
 - de valeurs (let)
 - de types (type)
 - d'exceptions (exception)



- Un module regroupe un ensemble de définitions apparentées
 - de valeurs (let)
 - de types (type)
 - d'exceptions (exception)
 - de sous-modules (module)



- Création d'un module

- Le mot clé **module** introduit la déclaration d'un nouveau module dont la définition est un bloc encadré par **struct** et **end**



- **Création d'un module**

- Le mot clé **module** introduit la déclaration d'un nouveau module dont la définition est un bloc encadré par **struct** et **end**
- Le bloc, appelé aussi une structure, est composé d'une suite de déclarations et/ou expressions.



- **Création d'un module**

- Le mot clé **module** introduit la déclaration d'un nouveau module dont la définition est un bloc encadré par **struct** et **end**
- Le bloc, appelé aussi une structure, est composé d'une suite de déclarations et/ou expressions.
- Les déclarations de modules peuvent être mêlées aux autres déclarations, et contenir elles-mêmes d'autres déclarations de modules.



- **Création d'un module**

Exemple :

```
# module Pile =  
  struct  
    type 'a t = 'a list ref  
    let create () = ref []  
    let push x p = p := x :: !p  
    let pop p = match !p with  
      | [] -> failwith "Pile vide"  
      | x :: r -> (p := r ; x)  
  end;;
```



- **Signature d'un module**

Exemple (suite) :

```
# module Pile :  
  sig  
    type 'a t = 'a list ref  
    val create : unit -> 'a list ref  
    val push : 'a -> 'a list ref -> unit  
    val pop : 'a list ref -> 'a  
  end
```



- Utilisation d'un module

- ① Module.(expression ou séquence utilisant le module)

Exemple

```
# String.(length "ab" + length "cde");;
```



- **Utilisation d'un module**

- ❶ Module.(expression ou séquence utilisant le module)

- Exemple**

- ```
String.(length "ab" + length "cde");;
```

- ❷ let open Module in expression ou séquence utilisant le module

- Exemple**

- ```
# let open String in length "abc";;
```



• Utilisation d'un module

- 1 Module.(expression ou séquence utilisant le module)

Exemple

```
# String.(length "ab" + length "cde");;
```

- 2 let open Module in expression ou séquence utilisant le module

Exemple

```
# let open String in length "abc";;
```

- 3 open Module;;
expression ou séquence utilisant le module;;

Exemple

```
# open String;;  
# length "abc";;
```



- **Restriction par une signature**

La construction (structure : signature)

- vérifie que la structure satisfait la signature
- rend inaccessibles les parties de la structure qui ne sont pas spécifiées dans la signature ;
- produit un résultat qui peut être lié par module $M = \dots$

module $X : S = M$

est équivalent à

module $X = (M : S)$



- **Notation with**

- La notation with permet d'ajouter des égalités de types dans une signature existante.
- Elle permet de créer facilement des signatures partiellement abstraites.

Exemple :

L'expression PLUS with type $t = \text{Euro.t}$ est une abréviation pour la signature

```
sig
  type t = Euro.t
  val plus : t -> t -> t
end
```



- Il est possible de définir des types de modules, c'est-à-dire de nommer des signatures, avec la syntaxe
module type Id = sig ... end

Exemple :

```
# module type Mon_type = sig
  type t
  val x : t
  val foo : t -> t
end;;
```



Types de module

Exemple :

```
# module N2 : Mon_type = struct
  type t = int
  let x = 1
  let y = 2
  let foo x = x + 1
end;;
module M : Mon_type
# M.y;;
Error : Unbound value "M.y"
```



Types de module

- Il est possible de définir un type de module à partir d'un module existant, grâce à la construction `module type of` :

Exemple :

```
# module type T = module type of M ; ;
```



Exemple : Un foncteur F paramétré par un module X de signature S

```
#module F(X : S) =
```

```
  struct
```

```
    type t = X.elt list array
```

```
    let create n = Array.create n []
```

```
    let add t x =
```

```
      let i = (X.hash x) mod (Array.length t) in
```

```
      t.(i) <- x :: t.(i)
```

```
    let mem t x =
```

```
      let i = (X.hash x) mod (Array.length t) in
```

```
      List.exists (X.eq x) t.(i)
```

```
  end;;
```



- Création d'un objet :

Exemple :

```
# object
```

```
val ch0 = v0 (* champ non mutable*)
```

```
val mutable ch1 = v1 (* champ mutable *)
```

```
val mutable ch2 = v2
```

```
method methode0 = (* ... *)
```

```
method methode1 = (* ... *)
```

```
end;;
```

NB : Les champs sont invisibles depuis l'extérieur.



- Appel de méthode :

- L'appel d'une méthode m par un objet o se fait par : **`o#m`**

Exemple :

```
# let mkpoint px py = object
  val mutable x = px
  val mutable y = py
  method sym_centre =
    x <- (-x);
    y <- (-y)
end;;
# let point_a = mkpoint 3 2;;
# point_a#sym_centre;;
```



- Appel de méthode :

- L'appel d'une méthode m par un objet o se fait par : **`o#m`**

Exemple :

```
# let mkpoint px py = object
  val mutable x = px
  val mutable y = py
  method sym_centre =
    x <- (-x);
    y <- (-y)
end;;
# let point_a = mkpoint 3 2;;
# point_a#sym_centre;;
```

- Les méthodes sont dynamiques (méthodes d'objet)



- Appel de méthode :

- L'appel d'une méthode m par un objet o se fait par : **o#m**

Exemple :

```
# let mkpoint px py = object
  val mutable x = px
  val mutable y = py
  method sym_centre =
    x <- (-x);
    y <- (-y)
end;;
```

```
# let point_a = mkpoint 3 2;;
# point_a#sym_centre;;
```

- Les méthodes sont dynamiques (méthodes d'objet)
 - La référence d'un objet à lui même se fait avec **self** (this en java), déclaré juste après object



- **Création d'une classe :**

Exemple :

```
# class mkpoint px py =  
  object (self)  
    val mutable x = px  
    val mutable y = py  
    method sym_x = y <- -y  
    method sym_y = x <- -x  
    method sym_center =  
      self#sym_x;  
      self#sym_y  
  end;;  
# let o = new mkpoint 8 (-1) in o#sym_center;;  
NB : Une classe n'a que des méthodes dynamiques
```



- **Syntaxe :**

```
class mere =
```

```
  object
```

```
    (* ... *)
```

```
  end
```

```
class fille =
```

```
  object
```

```
    inherit mere
```

```
    (* ... *)
```

```
  end
```



- **Exemple :**

```
# class compteur i =  
  object (self)  
    val mutable v = i  
    method valeur = v  
    method ajoute d = v <- v + d  
    method incremente = self#ajoute 1  
  end  
  
# class compteurPair i =  
  object (self)  
    inherit compteur (i / 2) as super  
    method decremente = self#ajoute (-1)  
    method valeur = 2 * super#valeur  
  end
```



- **Visibilité :**

- Une méthode est publique par défaut
- Une méthode privée est déclarée avec le mot clé `private` et n'est visible que dans la classe mère et les classes filles.
- Une méthode privée n'est utilisable que par les méthodes (ou le constructeur **initializer**) de la même instance.

Exemple :

```
#let po = object (self)
  val mutable x = 0
  method private ajoute d = x <- x + d
  method incremente = self#ajoute 1
end;;
```



- **Classe virtuelle :**

- Une méthode virtuelle est une méthode dont on ne donne pas la définition mais juste le type. Elle sera donnée dans les classes dérivées.



- **Classe virtuelle :**

- Une méthode virtuelle est une méthode dont on ne donne pas la définition mais juste le type. Elle sera donnée dans les classes dérivées.
- Une classe virtuelle est une classe qui contient une méthode virtuelle ; elle n'est pas instanciable.

Exemple :

```
# class virtual abstractCompteur i =  
  object (self)  
    val mutable x : int  
    method valeur = x  
    method incremente = self#ajoute 1  
    method virtual ajoute : int -> unit  
  end ; ;
```

